

Университет ИТМО

Расчётно-графическая работа №1.

Построение и визуализация фрактальных множеств

Участники:

Самсонова Владислава Р3210

Новикова Юлия Р3208

Исаева Александра-Ирина Р3209

Преподаватель практики:

Бойцев Антон Александрович

Санкт-Петербург, 2025

Задание 1.

Докажем свойства множеств Мандельброта.

Свойство №1. *Множество Мандельброта переходит само в себя при сопряжении. Иными словами, оно симметрично относительно вещественной оси*

Для доказательства первого свойства рассмотрим последовательности комплексных чисел, заданные следующим образом:

$$z_{n+1} = z_n^2 + c, \quad z_0 = 0, c \in \mathbb{C}$$

$$z'_{n+1} = z_n'^2 + \bar{c}, \quad z_0 = 0, \bar{c} \in \mathbb{C}$$

Сначала, используя метод математической индукции, покажем, что

$$z_n = \overline{z'_n}, \forall n:$$

1) База:

при $n = 1$:

$$z_1 = c, \quad z'_1 = \bar{c}, \quad z_1 = \overline{z'_1} = \bar{c}$$

2) Пусть при $n = k$ выражения справедливо, тогда покажем, что оно справедливо и для $n = k+1$:

$$z_k = \overline{z'_k} = y$$

$$z'_k = \overline{z_k} = \bar{y}$$

$$z_{k+1} = z_k^2 + c = y^2 + c$$

$$z'_{k+1} = z_k'^2 + \bar{c} = (\bar{y})^2 + \bar{c}$$

То есть для любого k справедливо:

$$z_{k+1} = \overline{z'_{k+1}}$$

Осталось показать, что $c \in M_n \Leftrightarrow \bar{c} \in M_n \boxtimes M_n \boxtimes$ множество Мандельброта:

$$\overline{z_n} = \overline{z'_n} = z'_n$$

Заметим, что:

$$|z_n| = |\bar{z}_n|$$

Отсюда следует, что последовательности z_n, z'_n либо сходятся, либо расходятся одновременно.

Получается, что при ограниченной последовательности z_n (z'_n), возникнет симметрия относительно вещественной части оси, так как $z_n = \bar{z}'_n, \forall n$

Свойство №2. Если $|c| \geq 2$, то c не принадлежит множеству Мандельброта.

Пусть $|c| = \delta + 2, \delta > 0$,

$$|z_{n+1}| = |z_n^2 + c| \geq |z_n|^2 - |c|$$

Заметим, что при $n \geq 2, |c| \geq 2$:

$$|z_n| > |c|$$

Тогда:

$$|z_{n+1}| \geq |z_n|^2 - |c| \geq |c|^2 - |c| = |c|(|c| - 1)$$

$$|c| - 1 = (2 + \delta) - 1 = 1 + \delta$$

$$|z_{n+1}| \geq |c|(1 + \delta)$$

Аналогично, запишем для z_{n+2} :

$$\begin{aligned} |z_{n+2}| &\geq |z_{n+1}|^2 - |c| \geq |c|^2(1 + \delta)^2 - |c| \\ &\geq |c|(1 + \delta)^2 \end{aligned}$$

Продолжая рассуждения получаем неравенство для каждого члена последовательности:

$$|z_{n+k}| = ||c|^k(\delta + 1)^k + c| \geq |c|(\delta + 1)^k$$

$$\lim_{k \rightarrow \infty} |z_{n+k}| = \lim_{k \rightarrow \infty} |c|(\delta + 1)^k = \infty$$

Получаем, что последовательность z_n не органичена при $|c| \geq 2$.

Задание 2.

Приведем код программы для построения множества Мандельброта.

```
import pygame
import os

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)

# ----- чтобы окно появлялось в верхнем левом углу -----
x = 20
y = 40
os.environ['SDL_VIDEO_WINDOW_POS'] = "%d,%d" % (x, y)
# -----

pygame.init()

W = 1200
H = 600

sc = pygame.display.set_mode((W, H))
pygame.display.set_caption("Множество Мандельброта - Используйте колесо мыши для масштабирования")
sc.fill(WHITE)

FPS = 30
clock = pygame.time.Clock()

# Параметры для построения фрактала
center_x, center_y = -0.5, 0 # центр просмотра
width, height = 3.0, 2.0 # размер области просмотра
n_iter = 100 # число итераций
zoom_factor = 1.2 # коэффициент масштабирования

# Функция для отрисовки фрактала
def draw_mandelbrot():
    sc.fill(WHITE)
    print("Начинаем построение множества Мандельброта...")

    for py in range(H):
        for px in range(W):
            # Преобразуем координаты пикселя в комплексную плоскость
            cx = center_x + (px - W / 2) * (width / W)
            cy = center_y + (py - H / 2) * (height / H)

            c = complex(cx, cy)
            z = complex(0)

            # Итерационный процесс
            for n in range(n_iter):
                z = z ** 2 + c
                if abs(z) > 2:
                    break

            # Раскрашиваем в зависимости от поведения
            if n == n_iter - 1:
                color = BLACK
            else:
                t = n / n_iter
                r = int(9 * (1 - t) * t * t * t * 255)
                g = int(15 * (1 - t) * (1 - t) * t * t * 255)
                b = int(8.5 * (1 - t) * (1 - t) * (1 - t) * t * 255)
                color = (r, g, b)

            sc.set_at((px, py), color)

    # Обновляем экран каждые 20 строк для визуализации прогресса
    if py % 20 == 0:
        pygame.display.update()
```

```

# Отображаем информацию
font = pygame.font.SysFont('Arial', 16)
info_text = f"Центр: ({center_x:.4f}, {center_y:.4f}) | Область: {width:.6f}x{height:.6f} | Итерации: {n_iter}"
text_surface = font.render(info_text, True, (255, 255, 255))
pygame.draw.rect(sc, (0, 0, 0), (10, 10, text_surface.get_width() + 10, text_surface.get_height() + 10))
sc.blit(text_surface, (15, 15))

pygame.display.update()
print("Построение завершено!")

# Первоначальная отрисовка
draw_mandelbrot()

# Главный цикл
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                running = False
            elif event.key == pygame.K_r: # Сброс к начальному виду
                center_x, center_y = -0.5, 0
                width, height = 3.0, 2.0
                n_iter = 100
                draw_mandelbrot()
            elif event.key == pygame.K_PLUS or event.key == pygame.K_EQUALS: # Увеличить итерации
                n_iter += 50
                draw_mandelbrot()
            elif event.key == pygame.K_MINUS: # Уменьшить итерации
                n_iter = max(50, n_iter - 50)
                draw_mandelbrot()
        elif event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 4: # Колесо мыши вверх - увеличение
                # Масштабируем к позиции мыши
                mouse_x, mouse_y = event.pos
                old_center_x = center_x
                old_center_y = center_y

                # Вычисляем координаты мыши в комплексной плоскости
                mouse_cx = center_x + (mouse_x - W / 2) * (width / W)
                mouse_cy = center_y + (mouse_y - H / 2) * (height / H)

                # Уменьшаем область просмотра
                width /= zoom_factor
                height /= zoom_factor

                # Пересчитываем центр так, чтобы точка под курсором осталась на месте
                center_x = mouse_cx - (mouse_x - W / 2) * (width / W)
                center_y = mouse_cy - (mouse_y - H / 2) * (height / H)

                # Увеличиваем количество итераций для большей детализации
                if width < 0.1:
                    n_iter = min(500, n_iter + 50)

                draw_mandelbrot()

            elif event.button == 5: # Колесо мыши вниз - уменьшение
                # Увеличиваем область просмотра
                width *= zoom_factor
                height *= zoom_factor

                # Уменьшаем количество итераций для ускорения
                if width > 1.0:
                    n_iter = max(100, n_iter - 50)

                draw_mandelbrot()

    elif event.type == pygame.MOUSEMOTION:
        # Перемещение с зажатой левой кнопкой мыши

```

```

if event.buttons[0]:
    # Вычисляем смещение
    rel_x, rel_y = event.rel
    center_x -= rel_x * (width / W)
    center_y -= rel_y * (height / H)
    draw_mandelbrot()

pygame.display.update()
clock.tick(FPS)

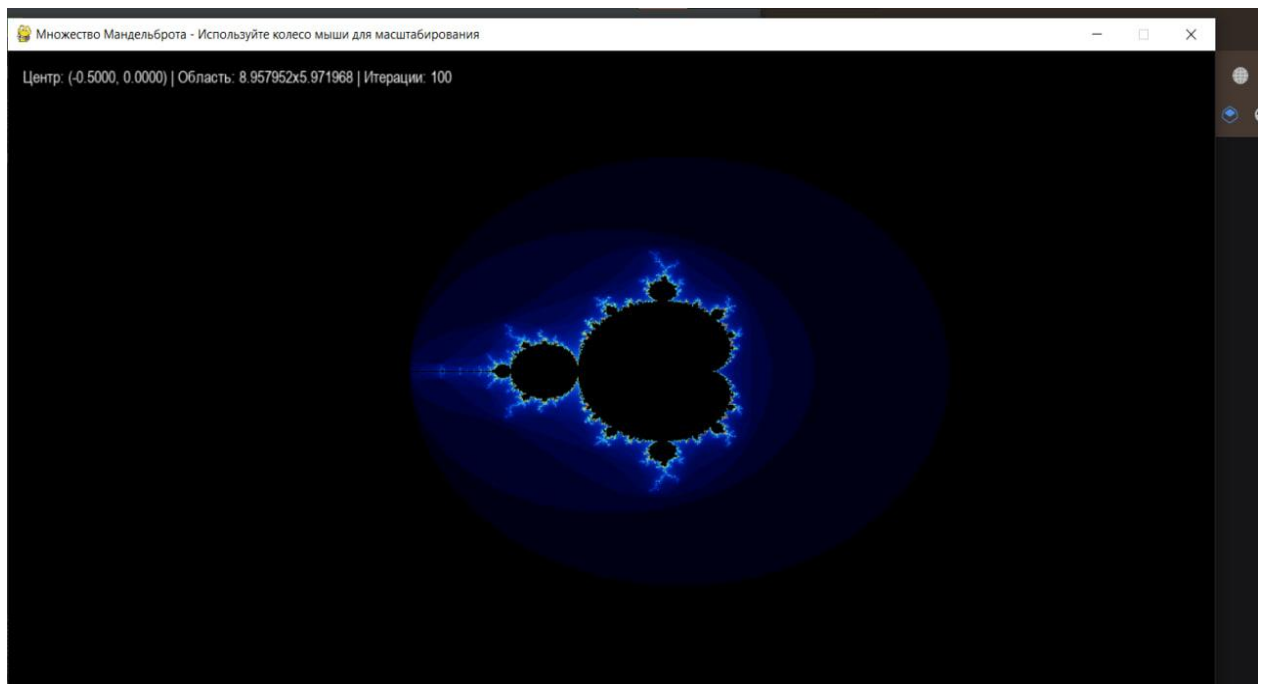
pygame.quit()

```

Изображения, построенные при разном числе итераций и приближении:

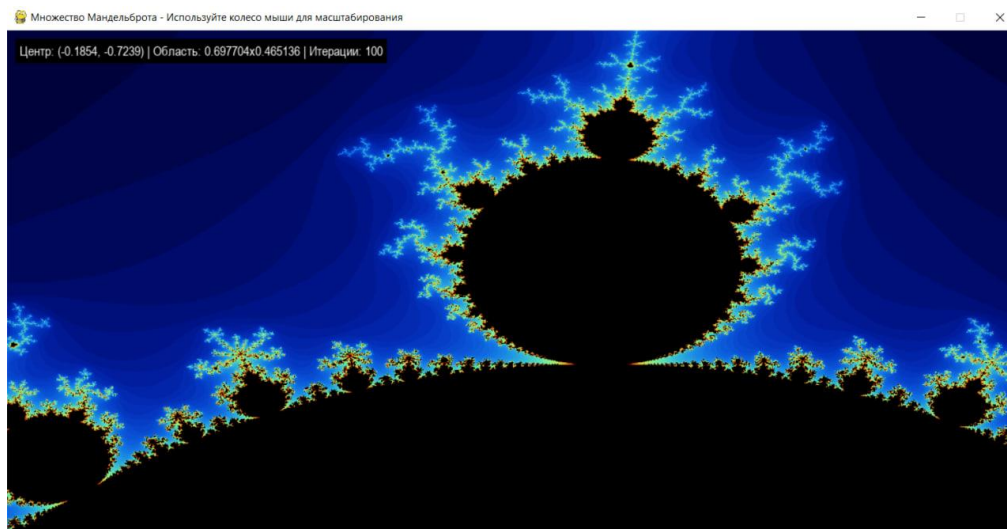
Далеко, 100 итераций:

Выбор масштаба и количества итераций обоснован желанием рассмотреть общую картину - то, как выглядит все множество. Сильная детализация (много итераций) здесь не нужна, так как она не будет заметна и отрисовка займет очень много времени.



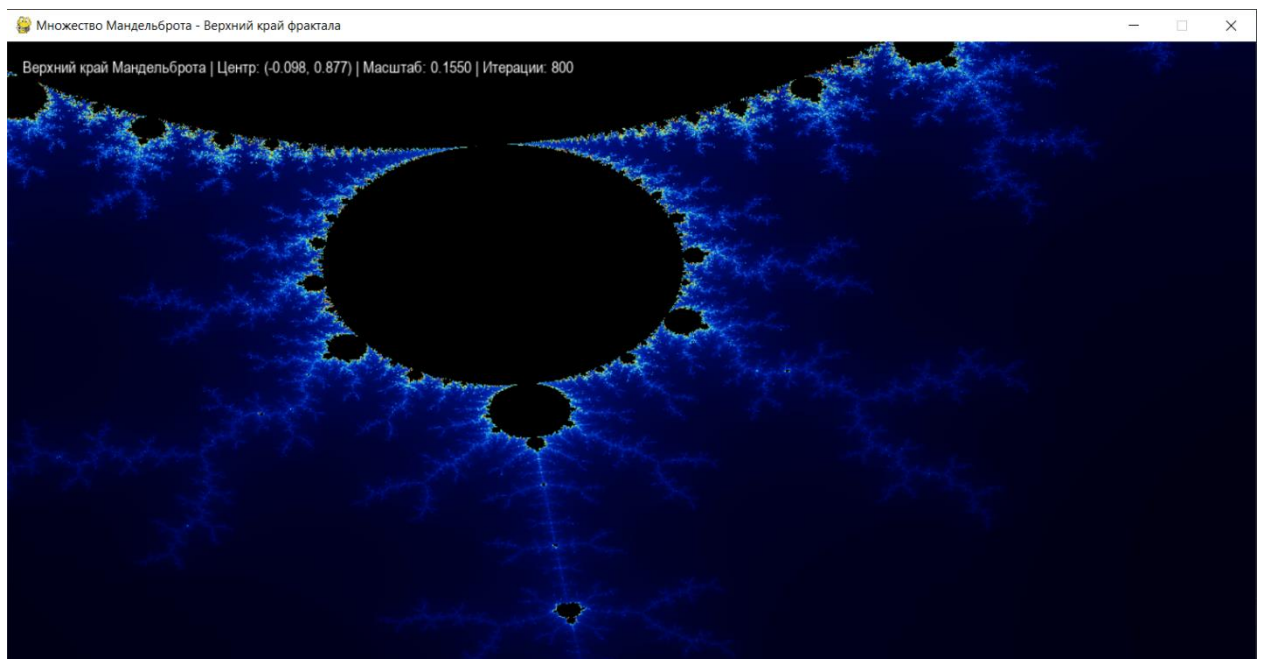
Близко, 100 итераций:

Такие параметры позволяют проследить фрактальную структуру множества, и, в то же время, заметить необходимость увеличения числа итераций при увеличении масштаба: видны нечеткие границы и “ступеньки”, ведь здесь малая плавность (четкость) прорисовки.



Очень близко, 800 итераций:

При такой детализации становится заметным, что существуют маленькие копии множества Мандельброта (в нижней части изображения). При этом, дальнейшее увеличение итераций даст незначительное улучшение при резком росте времени вычислений.



При приближении и увеличении количества итераций видим, что множество самоповторяется, а значит - имеет фрактальную структуру.

Задание №3.

Ниже представлен код для визуализации заполненных множеств Жюлиа:

```
import pygame
import os
import math

x, y = 20, 40
os.environ['SDL_VIDEO_WINDOW_POS'] = f'{x},{y}'
```



```

pygame.init()

class JuliaSet:
    def __init__(self, width=1000, height=700):
        self.W = width
        self.H = height
        self.sc = pygame.display.set_mode((self.W, self.H))
        self.c = complex(-0.8, 0.156)

        self.P = min(self.W, self.H) // 2
        self.scale = self.P / 0.25
        self.view = (0, 0)
        self.n_iter = 100
        self.color_scheme = lambda n: self.get_color(n)
        self.FPS = 30
        self.clock = pygame.time.Clock()
        self.running = True

    def get_color(self, n):
        if n == self.n_iter - 1:
            return (0, 0, 0) # Черный для точек множества
        else:
            r = (n % 2) * 32 + 128
            g = (n % 4) * 64
            b = (n % 2) * 16 + 128
            return (r, g, b)

    def calculate_escape_radius(self):
        r_c = abs(self.c)
        discriminant = 1 + 4 * r_c
        R = (1 + math.sqrt(discriminant)) / 2
        return R

    def generate_julia_set(self):
        self.sc.fill((255, 255, 255))

        R = self.calculate_escape_radius()
        print(f"c = {self.c}, Итерации: {self.n_iter}")
        for py in range(self.H):
            for px in range(self.W):
                a = (px - self.P + self.view[0]) / self.scale
                b = (py - self.P + self.view[1]) / self.scale

                z = complex(a, b)
                n = 0

                for n in range(self.n_iter):
                    z = z ** 2 + self.c
                    if abs(z) > R:
                        break

                color = self.color_scheme(n)
                self.sc.set_at((px, py), color)

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                self.running = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_PLUS or event.key ==
pygame.K_EQUALS:
                    self.scale *= 1.2
                    self.generate_julia_set()
                elif event.key == pygame.K_MINUS:

```

```

        self.scale /= 1.2
        self.generate_julia_set()
    elif event.key == pygame.K_UP:
        self.n_iter = min(self.n_iter + 50, 1000)
        self.generate_julia_set()
    elif event.key == pygame.K_DOWN:
        self.n_iter = max(self.n_iter - 50, 10)
        self.generate_julia_set()
    elif event.key == pygame.K_r:
        self.scale = self.P / 0.25
        self.view = (0, 0)
        self.n_iter = 100
        self.generate_julia_set()
    elif event.key == pygame.K_ESCAPE:
        self.running = False

def run(self):
    self.generate_julia_set()
    while self.running:
        self.handle_events()
        pygame.display.update()
        self.clock.tick(self.FPS)

pygame.quit()

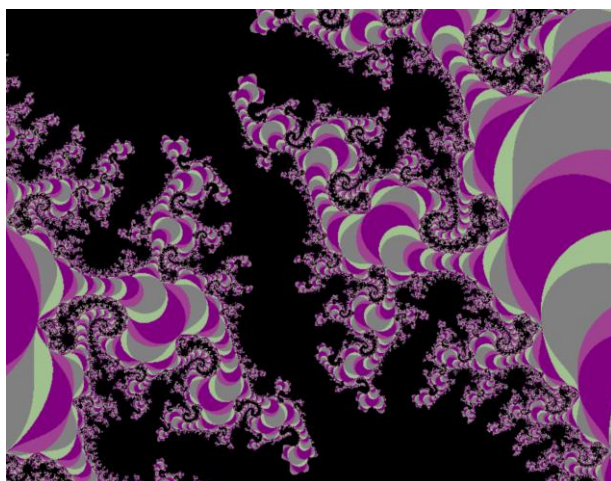
if __name__ == "__main__":
    julia = JuliaSet()
    julia.run()

```

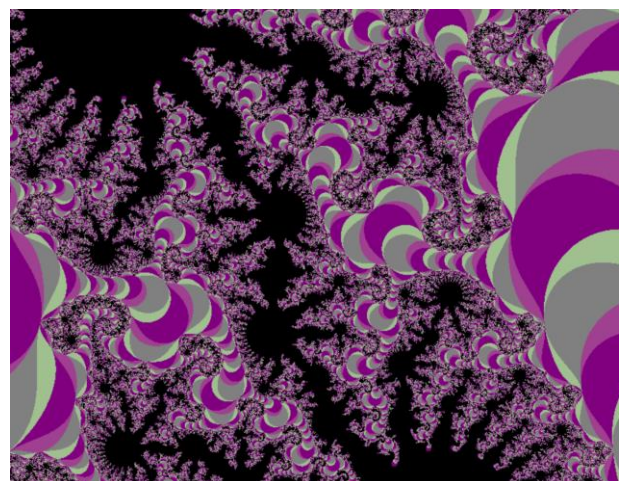
В коде, для определения устойчивости точки использовалось свойство Мандельберга №2, а для определения R неравенство:

$$R^2 - R \geq |c|$$

Для того чтобы проанализировать график на фракталы рассмотрим число $c = -0.8 + i0.156$:



100 итераций



150 итераций

Рис.3.1. Заполненное множество Жюлиа

При итерациях этого числа, появляются точки, для которых не выполняется свойство Мандельброта, то есть они перестают принадлежать

множеству Жюлиа (на графике эти точки черного цвета). Те точки, которые убежали на бесконечность при итерации, мы закрашиваем в другой цвет, он зависит от номера итерации. На *Рис. 3.1* видно, как с увеличением n расширяется закрашенная область и появляются новые фракталы, при максимальном количестве итераций (1000), на графике получаются бесконечно повторяющиеся узоры и изображение становится более детализированным:

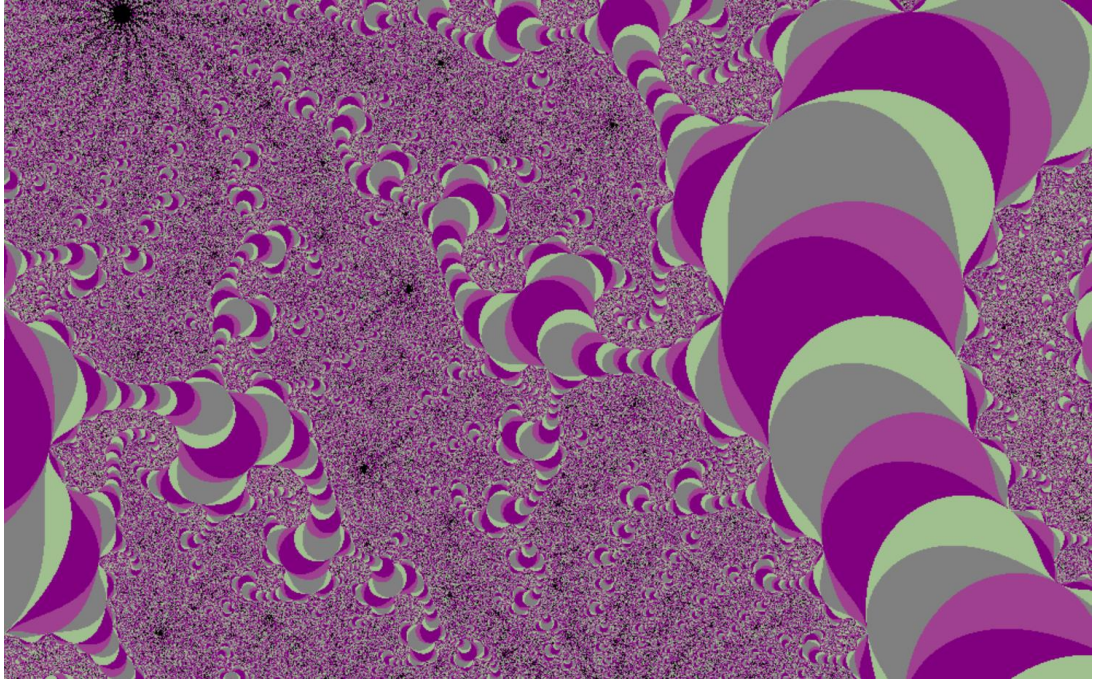
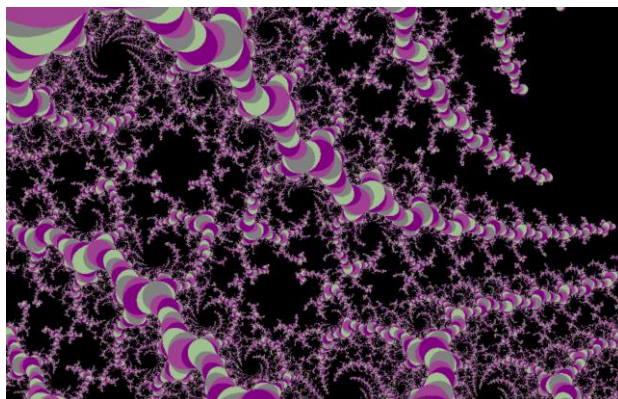
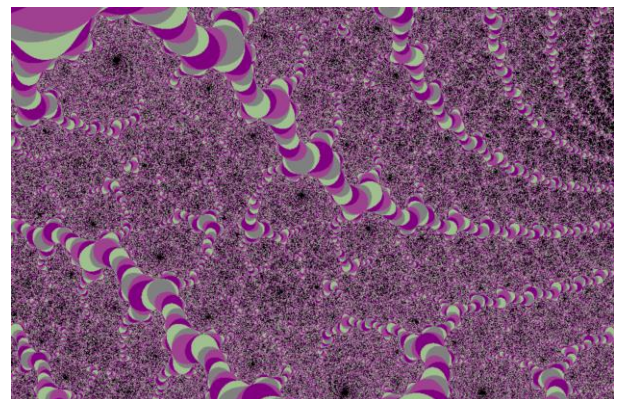


Рис. 3.2

На *Рис. 3.3* изображено множество для числа $c = -0.7 + i0.27$. Можно заметить, что заполненное множество Жюлиа стало меньше, потому что при итерировании исчезла часть неустойчивых точек и мы получили более точное приближение к этому множеству:



$n = 150$ итераций

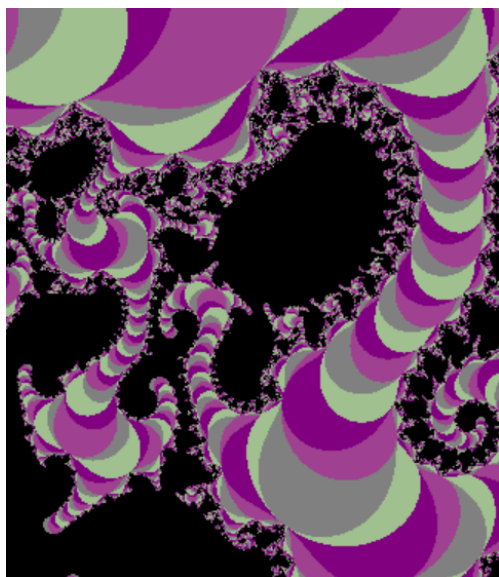


$n = 700$ итераций

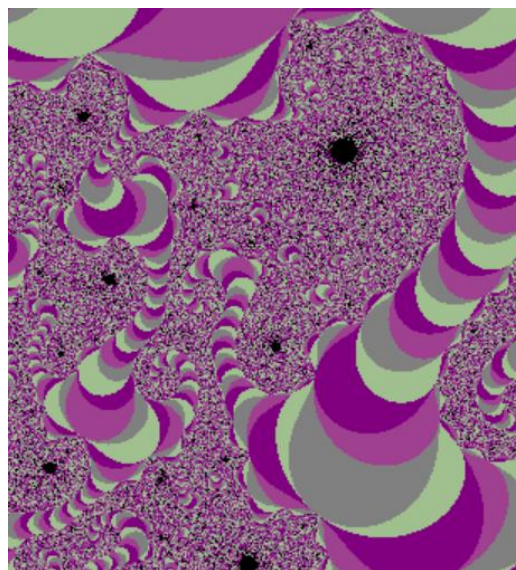
Рис. 3.3

Как уже было замечено, детализация фрактальной структуры и четкость границы заполненного множества (само множество Жюлиа)

напрямую зависят от количества отображений. Это также явно видно при приближении множества для числа $c = -0.74543 + i0.11301$:



N = 100 итераций



N = 600 итераций

Рис. 3.4

4. Фрактал кривая Гильберта

Структура

Кривая Гильберта — это непрерывная фрактальная кривая, заполняющая пространство, она проходит через все точки квадрата, не пересекается сама с собой и в пределе полностью заполняет двумерную область.

Её структура рекурсивна: каждый уровень фрактала состоит из четырёх уменьшенных копий самой кривой предыдущего порядка, расположенных в квадрате со стороной 2^n (n — номер итерации), повернутых на $\pm 90^\circ$ и соединённых таким образом, что кривая остаётся непрерывной.

Структура представляет собой самоподобную геометрическую конструкцию, в которой каждая предыдущая часть является уменьшенной версией последующего.

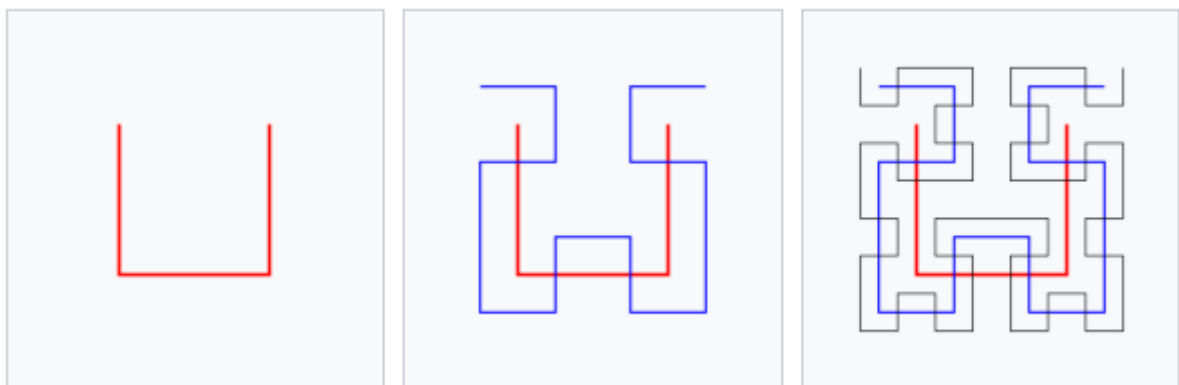


Рис. 4.1. Первый, второй и третий шаги построения кривой Гильберта

Построение

Создание кривой Гильберта для L-системы (системы Линденмайера)

Правила:

$$A \rightarrow - B F + A F A + F B -$$

$$B \rightarrow + A F - B F B - F A +$$

F означает «движение вперёд», «-» поворот влево на 90° , «+» поворот вправо на 90° , а A и B подставляются при рисовании.

Построение основано на рекурсивном делении квадрата на четыре равные части и соединении их последовательностью линий, образующих путь через все квадраты.

- На первой итерации кривая состоит из трёх отрезков, соединяющих четыре точки в виде буквы «U».
- На второй итерации каждая сторона «U»-образной фигуры “заменяется” уменьшенной копией исходной кривой, ориентированной под разными углами.
- На третьей и последующих итерациях процесс повторяется: каждый отрезок предыдущей итерации “заменяется” повернутой и масштабированной копией кривой более низкого уровня.

Код для программной визуализации

```
def hilbert(level, angle = 90, step = 10):
    if level == 0:
        return
    turtle.right(angle)
    hilbert(level - 1, -angle, step)
    turtle.forward(step)
    turtle.left(angle)
    hilbert(level - 1, angle, step)
    turtle.forward(step)
    hilbert(level - 1, angle, step)
    turtle.left(angle)
    turtle.forward(step)
    hilbert(level - 1, -angle, step)
    turtle.right(angle)
```

```
turtle.speed(0)
hilbert(4)
turtle.done()
```

Функция описывает пошаговый алгоритм построения самоподобного фрактала, где каждый вызов рисует одну из четырёх частей фигуры и соединяет их под правильными углами.

При вызове `hilbert(4, 90, 10)` выполняется 4 уровня рекурсии, что создаёт детализированный рисунок кривой Гильберта, полностью заполняющей квадратную область 16x16 (рис. 4.2).

ур.	что происходит	поворот перед вызовом	действия внутри уровня	что происходит после
4	Запуск основного уровня — вся кривая Гильберта	<code>turtle.right(90)</code>	Вызывает <code>hilbert(3, -90)</code> → двигается → поворачивает → вызывает <code>hilbert(3, 90)</code> → двигается → <code>hilbert(3, 90)</code> → двигается → <code>hilbert(3, -90)</code>	Возвращает управление (рисунок закончен)
3	Рекурсивная подфигура — ¼ части кривой	<code>turtle.right(90)</code> от уровня 4	Вызывает <code>hilbert(2, -90)</code> → шаг → <code>hilbert(2, 90)</code> → шаг → <code>hilbert(2, 90)</code> → шаг → <code>hilbert(2, -90)</code>	Возвращает координату на следующую соединяющую линию
2	Более мелкий фрагмент — ¼ части уровня 3	<code>turtle.right(90)</code> от уровня 3	Рекурсивно вызывает <code>hilbert(1, -90)</code> → шаг → <code>hilbert(1, 90)</code> → шаг → <code>hilbert(1, 90)</code> → шаг → <code>hilbert(1, -90)</code>	Возвращает позицию
1	Простейшая форма кривой — «U»-образная фигура	<code>turtle.right(90)</code>	Вызывает <code>hilbert(0, -90)</code> (ничего не делает) → шаг → <code>hilbert(0, 90)</code> → шаг → <code>hilbert(0, 90)</code> → шаг → <code>hilbert(0, -90)</code>	Завершает рисование мини-"U"

Визуализация

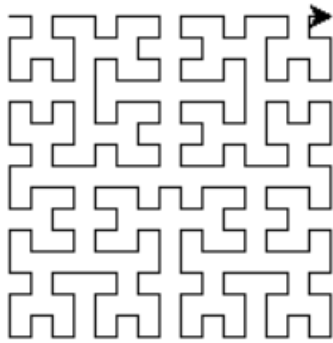


Рис. 4.2 Программная визуализация кривой Гильберта уровня 4, заполняющая квадрат 16x16

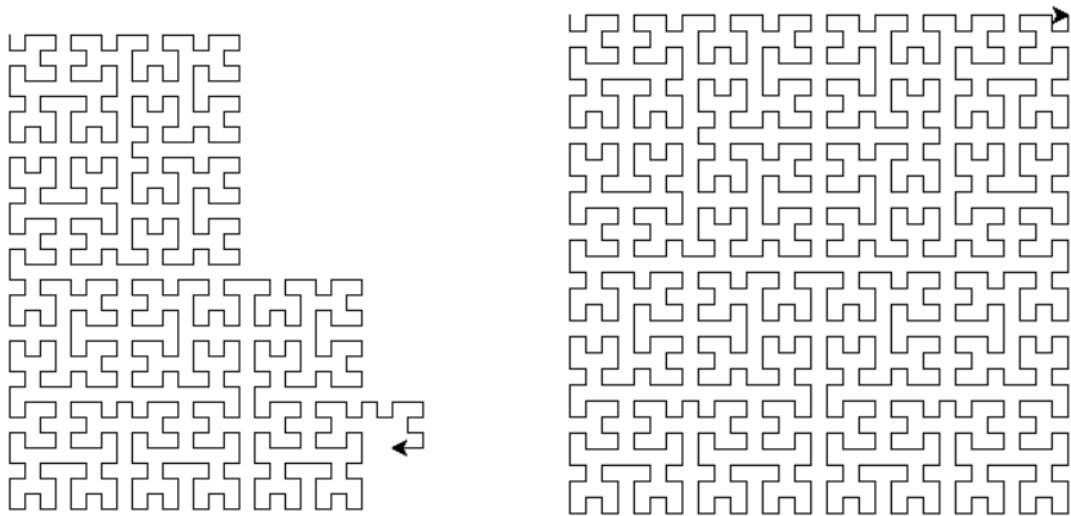


Рис. 4.3 и 4.4. Программная визуализация кривой Гильберта 5 уровня, заполняющая квадрат 32x32. Слева - процесс заполнения.